

MiniANN: C++ Artificial Neural Network Library

Project Report — Architecture, Features, OOP Mapping and UML Diagrams

MiniANN Project (Problem Statement 5, Spec v1.3)

October 4, 2026

What this report is. A self-contained technical report on the MiniANN codebase: what it does, how it is organised, which features it provides, and — in detail — where each Object-Oriented Programming (OOP) concept is used (with file references and class names). UML-style class diagrams are drawn directly in $\text{\LaTeX}$ with TikZ so the document compiles with a plain `pdf $\text{\LaTeX}$` installation. A complete concrete mathematical trace for one training dry run is detailed, and recent system extensions (including the RMSprop optimiser, ELU activation, L_2 weight decay, gradient norm clipping, and the interactive 3D OpenGL Network Viewer) are fully documented.

Contents

1	Project Overview	3
1.1	Design rules enforced by the codebase	3
1.2	Directory layout	3
2	Objectives and Scope	4
2.1	Objectives	4
2.2	Out of scope (by design)	4
3	System Architecture	4
3.1	Training pipeline (six steps)	5
4	Features in Detail	5
4.1	Network construction	5
4.2	Activation functions (8)	5
4.3	Loss functions (3)	6
4.4	Optimisers (4)	6
4.5	Training engine & regularisation	6
4.6	Datasets and preprocessing	6
4.7	Metrics, persistence and export	7
4.8	Interactive 2D & 3D visual front-ends	7
5	Technology and Build	7
6	Module Reference (File by File)	8
7	OOP in MiniANN — Concept-by-Concept Mapping	8
7.1	Detailed Analysis of Selected Patterns	8
7.1.1	Strategy Pattern (<code>IOptimizer</code> , <code>IActivation</code> , <code>INormalizer</code>)	8
7.1.2	Observer Pattern (<code>TrainingCallback</code> , <code>LiveBridge</code>)	8
7.1.3	Visitor Pattern (<code>INetworkVisitor</code>)	8
8	UML / Class Diagrams	11

9	Dynamic View & Concrete Dry Run Trace	13
9.1	Per-batch training sequence	13
9.2	Concrete mathematical trace for 1 dry run	13
9.2.1	Step 1: Forward Propagation	14
9.2.2	Step 2: Backward Propagation	14
9.2.3	Step 3: RMSprop Parameter Update	14
10	Verification and Test Results	14
11	Minimal Usage Example	15
12	Conclusion	16

1 Project Overview

MiniANN is a modular, lightweight feed-forward Artificial Neural Network framework written from scratch in modern C++17/C++20. It deliberately uses no PyTorch, TensorFlow, Eigen, BLAS, or plotting libraries. The goal is architectural clarity, strict encapsulation, and clean object-oriented design — not raw execution speed.

A user can:

1. construct arbitrary layer topologies as `NeuralNetwork` $\rightarrow$ `Layer` $\rightarrow$ `Neuron`,
2. swap mathematical pieces at runtime through pure-virtual interfaces (`IActivation`, `ILoss`, `IOptimizer`, `IMetric`, `ILogger`, `IVisualizer`, `ILRScheduler`),
3. train with mini-batch backpropagation (`Trainer::fit`) using Xavier/He initialisation, with support for L_2 weight decay, gradient norm clipping, and learning rate scheduling,
4. load data from built-in logic gates or CSV files (`Dataset::loadCSV`, train/validation/test splits),
5. evaluate binary and multi-class problems (threshold rule for one output, argmax otherwise, per-class precision/recall/F1),
6. persist models as human-readable text (MINIANN 1/2 format, 17-digit IEEE-754 precision),
7. export loss curves and self-contained HTML reports with SVG charts,
8. inspect network operations in real time through an interactive desktop workbench featuring both 2D analytics and a hardware-accelerated 3D OpenGL Network Viewer,
9. prove correctness with gradient checks, serialisation round-trips, and AND/OR/XOR + Iris demos.

Spec version: **1.3** / **Problem Statement 5**. Language standard: **C++17/C++20** with `-Wall -Wextra -Wpedantic`, no raw `new/delete`, const-correctness, and the naming convention PascalCase classes / I-prefixed interfaces / camelCase methods / trailing `_` members.

1.1 Design rules enforced by the codebase

- `derivative(z)` takes the pre-activation z , not the output a .
- `Neuron::backward()` accumulates (`+=`); gradients are zeroed once per batch via `zeroGradients()`.
- `Layer::backward()` reads weights before the optimiser step.
- Serialiser tags (`sigmoid`, `tanh`, `relu`, `leaky_relu`, `swish`, `elu`, `linear`, `softmax`) match `ActivationFactory::create()` exactly.
- The network never trains itself; `Trainer` + `IOptimizer` own mutation.

1.2 Directory layout

```
1 MiniANN_MVP /
2   CMakeLists.txt           # miniann lib + demos + tests + optional Qt GUI
3   build.bat / build.ps1    # Native g++ build scripts (two-stage: lib once)
4   README.md docs/          # Technical architecture specifications
5   include/miniann/         # Frozen interface contracts (headers)
6       types.hpp             # Vector, Matrix aliases
7       activation.hpp        # IActivation x8 + ActivationFactory
8       neuron.hpp            # Neuron, WeightInit, gradWeights/gradBias
9       layer.hpp             # Layer container, joint-softmax path
10      network.hpp           # NeuralNetwork container, visitor traversal
```

```

11  loss.hpp                # ILoss x3 + LossFactory
12  optimizer.hpp          # IOptimizer x4 + OptimizerFactory +
    OptimizerConfig
13  trainer.hpp            # TrainingConfig, TrainingHistory, Trainer
14  scheduler.hpp          # ILRScheduler x3 + LRSchedulerFactory
15  dataset.hpp            # Dataset container, splitTrainValTest
16  preprocessing.hpp      # normalizeMaxAbs, oneHotEncode
17  normalize.hpp          # INormalizer x3 + NormalizerFactory
18  validate.hpp           # IValidator + CompositeValidator
19  metrics.hpp            # IMetric, Accuracy, ConfusionMatrix
20  logger.hpp             # LogLevel, ILogger, ConsoleLogger, FileLogger
21  serializer.hpp         # ModelSerializer, CSVLossExporter
22  visualizer.hpp         # IVisualizer x6 + LiveConsole + HtmlReport
23  report.hpp             # ConfigSection, SummarySection, HtmlReportBuilder
24  datasource.hpp         # IDatasetSource x5 + ITargetSelector chain
25  experiment.hpp         # ExperimentConfig, PreparedData, Controller
26  cli.hpp               # Header-only CLI parsing
27  miniann.hpp           # Single umbrella include
28  src/                  # One .cpp implementation per header
29  demos/                # CLI demos and visualizers
    xor_demo.cpp and_or_demo.cpp iris_demo.cpp compare_demo.cpp
31  playground.cpp early_stop_demo.cpp gui_qt.cpp
32  network3d/            # 3D OpenGL Network Viewer module
    network3d_widget.hpp/.cpp camera3d.hpp layout3d.hpp edge_budget.hpp
33  tests/                # Automated unit & integration verification
    test_basic.cpp test_choices.cpp gradient_check.cpp
36  test_training.cpp test_correctness.cpp test_callbacks.cpp
37  test_patterns.cpp test_viz3d_math.cpp
38  data/                 # Synthetic and benchmark datasets (iris_small.csv)

```

2 Objectives and Scope

2.1 Objectives

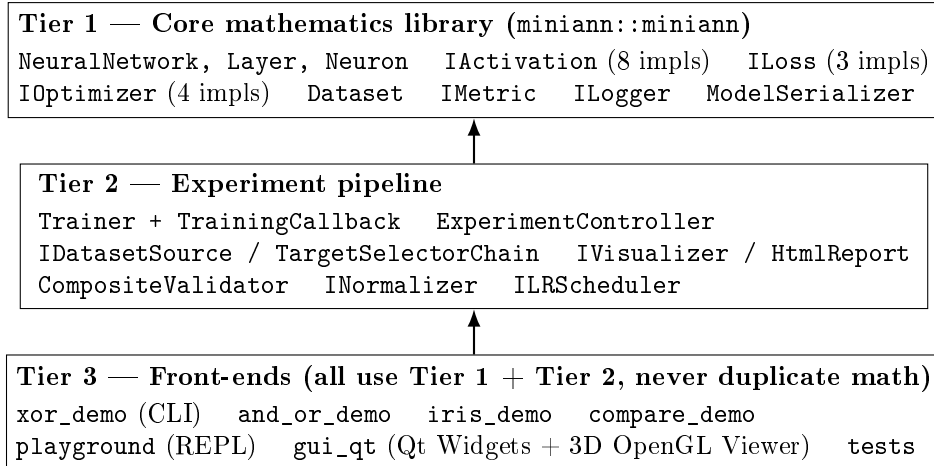
1. Provide a readable, from-scratch neural-network implementation in which every backpropagation step is explicit (explicit loops, cached `lastInputs_`/`lastZ_`).
2. Demonstrate textbook OOP: abstraction, inheritance, polymorphism, encapsulation, composition, plus the Strategy, Observer, Factory, Builder, Chain-of-Responsibility, Visitor, Composite, Façade, and Memento patterns.
3. Support real experiments without editing C++: CLI flags, an interactive REPL playground, CSV ingestion, and an interactive Qt GUI with 2D analytics and 3D spatial rendering over the same engine.
4. Make every claim verifiable: gradient check (finite differences with relative error $< 10^{-7}$), serialisation round-trip, deterministic splits, and loss/accuracy artefacts (CSV + HTML).

2.2 Out of scope (by design)

Convolutional/recurrent networks, GPU execution, and dynamic autograd graphs. The framework concentrates on feed-forward multi-layer perceptrons, transparent dense backpropagation, and rigorous architectural design.

3 System Architecture

MiniANN is organised as a clean three-tier system:



Ownership rule: `NeuralNetwork` owns forward/backward chaining only. `Trainer + IOptimizer` own the training policy and parameter mutation. `Dataset` owns data. `ILoss / IActivation / IMetric / ILogger` are pure interfaces. The UI never constructs datasets, networks, losses, or optimisers directly; it populates one `ExperimentConfig` and hands it to `ExperimentController`.

3.1 Training pipeline (six steps)

1. **Data:** `Dataset::validate()` (dimensions, NaN/Inf), optional deterministic splitting (`splitTrainValTest()`); normalisation strategies via `INormalizer` and polymorphic sourcing via `IDatasetSource`.
2. **Build:** `NeuralNetwork` assembles `Layer` objects of `Neuron` units; `ActivationFactory::create(name)` supplies the non-linearity; initialisation strategies apply He or Xavier limits.
3. **Train:** `Trainer::fit()`: shuffle with mt19937 seed, mini-batch partition, per-batch `zeroGradients()`, per-sample `predict` $\rightarrow$ `loss.gradient()` $\rightarrow$ `backward()` (accumulates `gradW += delta*a`).
4. **Regularise & Clip:** Add L_2 penalty to weight gradients ($\mathbf{g}_W += \lambda \mathbf{W}$) and rescale global gradient norm if $\|\mathbf{g}\|_2 > g_{\max}$.
5. **Optimise:** Execute `optimizer.step(net, B)` with learning rate scheduling.
6. **Evaluate & Persist:** Compute accuracy and confusion matrices; stream metrics to observers; persist weights via `ModelSerializer` or export standalone HTML reports.

4 Features in Detail

4.1 Network construction

Networks are composed, not inherited: `NeuralNetwork` holds `std::vector<Layer>`, and each `Layer` holds `std::vector<Neuron>`. Topology dimensions are strictly validated; invalid layer dimensions throw `std::invalid_argument`. Hidden layers accept a `fanOut` parameter to scale Xavier/He variance limits accurately.

4.2 Activation functions (8)

`IActivation` declares `activate(z)`, `derivative(z)` (at pre-activation z), `name()`, and `isVector()` (false except Softmax). Concrete classes:

- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$, with derivative $\sigma(z)(1 - \sigma(z))$.

- **Tanh:** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, with derivative $1 - \tanh^2(z)$.
- **ReLU:** $\max(0, z)$, with derivative 1 for $z > 0$, 0 otherwise.
- **LeakyReLU:** $\max(\alpha z, z)$ with validated $\alpha \in (0, 1)$, default 0.01.
- **Swish:** $z \cdot \sigma(z)$, with smooth analytic derivative $\sigma(z) + z\sigma(z)(1 - \sigma(z))$.
- **ELU (Exponential Linear Unit):** z for $z > 0$, and $\alpha(e^z - 1)$ for $z \leq 0$ ($\alpha = 1.0$), avoiding dying neurons by allowing negative saturation.
- **Linear:** $f(z) = z$, $f'(z) = 1$ for regression output heads.
- **Softmax:** Joint stable operator $\frac{e^{z_i - \max(z)}}{\sum e^{z_j}}$ evaluated at layer level with Jacobian backpropagation.

4.3 Loss functions (3)

`ILoss` declares `compute(pred, target)` and `gradient(pred, target)`.

- **MSELoss:** Mean Squared Error $\frac{1}{2n} \sum (y_i - \hat{y}_i)^2$, default for regression and general heads.
- **BCELoss:** Binary Cross-Entropy with numerical clamping $[\epsilon, 1 - \epsilon]$, rejecting multi-output targets.
- **CCELoss:** Categorical Cross-Entropy for one-hot multi-class classification, ensuring proper probability penalisation.

4.4 Optimisers (4)

`IOptimizer` declares `step(net, batchSize)`, `visit(visitor)`, and `name()`.

- **SGD:** Standard mini-batch gradient descent $\theta \leftarrow \theta - \frac{\eta}{B}g$.
- **Momentum:** Classical momentum maintaining velocity buffer $v \leftarrow \mu v - \frac{\eta}{B}g$.
- **Adam:** Adaptive moment estimation tracking first (m_t) and second (v_t) moments with bias correction $\hat{m}_t, \hat{v}_t$.
- **RMSprop:** Exponentially decaying average of squared gradients $v_t \leftarrow \alpha v_{t-1} + (1 - \alpha)g_t^2$, dividing updates by $\sqrt{v_t + \epsilon}$.

4.5 Training engine & regularisation

`Trainer::fit()` manages the epoch loop. Key capabilities include:

- **L_2 Weight Decay:** Adds λW to gradients prior to optimiser step to penalise large weights.
- **Gradient Norm Clipping:** Computes total Frobenius norm $\|g\|_2 = \sqrt{\sum \|g_W\|_F^2 + \sum \|g_b\|_2^2}$ and rescales gradients if $\|g\|_2 > g_{\max}$.
- **Learning Rate Schedulers:** Modulates η per epoch using `StepLR` or `ExponentialLR`.

4.6 Datasets and preprocessing

`Dataset` stores parallel inputs and targets. Supports CSV ingestion (`Dataset::loadCSV`), RFC-4180 parsing, and deterministic 3-way partitioning (`splitTrainValTest`). Preprocessing includes `ZScoreNormalizer`, `MinMaxNormalizer`, and one-hot target encoding.

4.7 Metrics, persistence and export

- **Metrics:** `Accuracy` and `ConfusionMatrix` providing per-class precision, recall, and F1-score.
- **Persistence:** `ModelSerializer` writes human-readable MINIANN 1/2 formats at 17-digit precision, capturing layer topology, weights, and optimiser momentum states.
- **Export:** `CSVLossExporter` exports training trajectories; `HtmlReportBuilder` generates standalone interactive HTML reports with inline SVG charts.

4.8 Interactive 2D & 3D visual front-ends

- **CLI Demos:** `xor_demo`, `and_or_demo`, `iris_demo`, `compare_demo`, and the interactive REPL playground.
- **Qt6 Workbench** (`gui_qt`): Real-time training dashboard with loss curves, accuracy plots, decision boundary heatmaps, confusion matrix modal, and dataset partition cards.
- **3D OpenGL Network Viewer:** Integrated hardware-accelerated 3D visualisation module (`Network3DWidget`) with interactive orbit/pan/zoom camera, dynamic edge magnitude thresholding, layer focus isolation, and individual neuron picking.

5 Technology and Build

- **Language Standard:** ISO C++17 / C++20, standard library only for core engine.
- **Compiler Support:** MSYS2 UCRT64 / MinGW g++ 13+, Clang, and MSVC.
- **Build Orchestration:** Two-stage native compilation via `build.bat` / `build.ps1` (compiles core object files into `obj/` once, then incrementally links targets). CMake 3.14+ build configuration also provided.
- **Quality Gates:** Zero compiler warnings with `-Wall -Wextra -Wpedantic`, memory safety via RAII, and verified numeric stability.

6 Module Reference (File by File)

7 OOP in MiniANN — Concept-by-Concept Mapping

Table 1 provides a comprehensive summary mapping core OOP principles and GoF design patterns directly to their concrete implementations in the MiniANN codebase.

7.1 Detailed Analysis of Selected Patterns

7.1.1 Strategy Pattern (`IOptimizer`, `IActivation`, `INormalizer`)

The training engine relies on the Strategy pattern to decouple optimisation algorithms from the backpropagation traversal. The `Trainer` holds an `IOptimizer&` reference. When `optimizer.step(net, batchSize)` is invoked, polymorphism directs execution to the selected algorithm (e.g., `RMSprop::step` or `Adam::step`), updating parameters without branching logic inside the trainer.

7.1.2 Observer Pattern (`TrainingCallback`, `LiveBridge`)

During iterative training, decoupling telemetry from computational math is paramount. `Trainer::fit` accepts an optional `TrainingCallback*` observer. At the conclusion of each epoch, it invokes `callback->onEpoch(epoch, history)`. Concrete observers such as `LiveConsole` render ASCII curves, while the Qt workbench’s bridge queues thread-safe snapshots for 2D and 3D rendering.

7.1.3 Visitor Pattern (`INetworkVisitor`)

To perform non-intrusive operations across the network structure (such as serialisation formatting, statistics collection, or parameter decay), `NeuralNetwork` implements `accept(INetworkVisitor& visitor)`. The visitor visits each layer and neuron sequentially, separating operational algorithms from the network data structure.

Header	Key Types	Architectural Role
types.hpp	Vector, Matrix	Standard type aliases over <code>std::vector<double></code>
activation.hpp	IActivation, 8 impls, ActivationFactory	Non-linear activation functions and derivatives
neuron.hpp	Neuron, WeightInit	Weight/bias state, forward/backward accumulation
layer.hpp	Layer	Layer container, Xavier initialisation, softmax routing
network.hpp	NeuralNetwork	Network container, forward/backward chaining, visitor
loss.hpp	ILoss, 3 impls, LossFactory	Loss functions and analytic output gradients
optimizer.hpp	IOptimizer, 4 impls, OptimizerFactory	Weight update policies (SGD, Momentum, Adam, RMSprop)
trainer.hpp	Trainer, TrainingConfig, TrainingHistory	Mini-batch training loop, L_2 decay, gradient clipping
scheduler.hpp	ILRScheduler, 3 impls, factory	Dynamic learning rate schedule policies
dataset.hpp	Dataset, logic gate makers	In-memory data store, CSV parsing, 3-way splits
normalize.hpp	INormalizer, ZScore, MinMax	Feature normalisation strategies
validate.hpp	IValidator, CompositeValidator	Pre-flight dataset validation rules
metrics.hpp	IMetric, Accuracy, ConfusionMatrix	Evaluation metrics, multi-class performance statistics
logger.hpp	ILogger, ConsoleLogger, FileLogger	Decoupled logging abstractions
serializer.hpp	ModelSerializer, CSVLossExporter	Binary/text model persistence and CSV exporting
visualizer.hpp	IVisualizer, 6 impls, HtmlReport	Text/SVG terminal visualisers and HTML reporters
report.hpp	ConfigSection, SummarySection, builder	Fluent builder for standalone HTML experiment reports
datasource.hpp	IDatasetSource, TargetSelectorChain	Polymorphic data sources and label election chain
experiment.hpp	ExperimentConfig, ExperimentController	Unified Faade controller linking UI to engine
network3d_widget	Network3DWidget, Camera3D, Layout3D	Hardware-accelerated 3D OpenGL network renderer

Table 1: MiniANN Module Reference and Responsibilities

Pillar / Pattern	Code Location	Mechanism & Implementation Details
Abstraction	activation.hpp:10, loss.hpp:9, optimizer.hpp:10, trainer.hpp:31, datasource.hpp:75	Pure-virtual interfaces (<code>activate</code> , <code>compute</code> , <code>step</code> , <code>onEpoch</code> , <code>load</code>). Consumers program strictly to interface contracts.
Inheritance	Sigmoid, Tanh, ELU : <code>IActivation</code> ; SGD, Adam, RMSprop : <code>IOptimizer</code> ; MSELoss : <code>ILoss</code>	Public interface inheritance establishing clear is-a relationships across all mathematical and operational families.
Polymorphism	optimizer->step(), loss->gradient(), act->derivative(), visual->render()	Dynamic runtime dispatch via vtables; factories return base pointers manipulated without knowledge of concrete types.
Encapsulation	Neuron, Layer, NeuralNetwork, Trainer	Internal states (weights, biases, cached activations, momentum tables) are <code>private</code> ; mutated only via disciplined methods.
Composition	NeuralNetwork has Layer has Neuron; HtmlReport has IHtmlSection	Lifetimes flow downward; composite containers own elements by value or <code>unique_ptr</code> .
SRP	Dataset, Trainer, ModelSerializer, CSVLossExporter	Single responsibility per class: Dataset owns storage, Trainer owns training, ModelSerializer owns file schemas.
Open/Closed	IActivation, IOptimizer, IVisualizer, IHtmlSection	Open for extension, closed for modification: adding ELU or RMSprop required zero edits to existing activation/optimizer classes.
Strategy	IOptimizer, ILoss, IActivation, INormalizer, ILRScheduler	Interchangeable algorithmic strategies swapped dynamically at runtime through configuration keys.
Observer	TrainingCallback, LiveConsole, LiveBridge	Training loop broadcasts epoch events to registered observers without coupling training math to presentation.
Factory	ActivationFactory, OptimizerFactory, LossFactory, NormalizerFactory	Maps runtime strings to heap-allocated objects wrapped in smart pointers; centralises instantiation logic.
Builder	HtmlReportBuilder	Fluent interface (<code>.withSeries().withBoundary().build()</code>) constructing complex HTML reports step by step.
Chain of Resp.	TargetSelectorChain	Successively attempts header name, distinct class count, and fallback column links to elect target labels.
Visitor	INetworkVisitor, NeuronInspectorVisitor	Traverses network hierarchy to perform inspections or weight adjustments without polluting domain models.
Composite	CompositeValidator, CallbackList	Treats individual validation rules and collections uniformly by aggregating child nodes under the same interface.
Façade	ExperimentController	Unified entry point (<code>prepare</code> , <code>run</code>) shielding client code and GUI from internal subsystem complexity.
Bridge	LiveBridge	Thread-safe producer-consumer bridge with mutexes and atomics decoupling background training from the GUI.
Memento	ModelSerializer	Captures and restores network states (weights, biases, momentum vectors) without exposing internal invariants.
RAII / Safety	Entire codebase	Ownership strictly expressed via <code>std::unique_ptr</code> and <code>std::shared_ptr</code> ; zero raw memory leaks; typed exceptions.

Table 2: Comprehensive OOP Mapping in MiniANN

8 UML / Class Diagrams

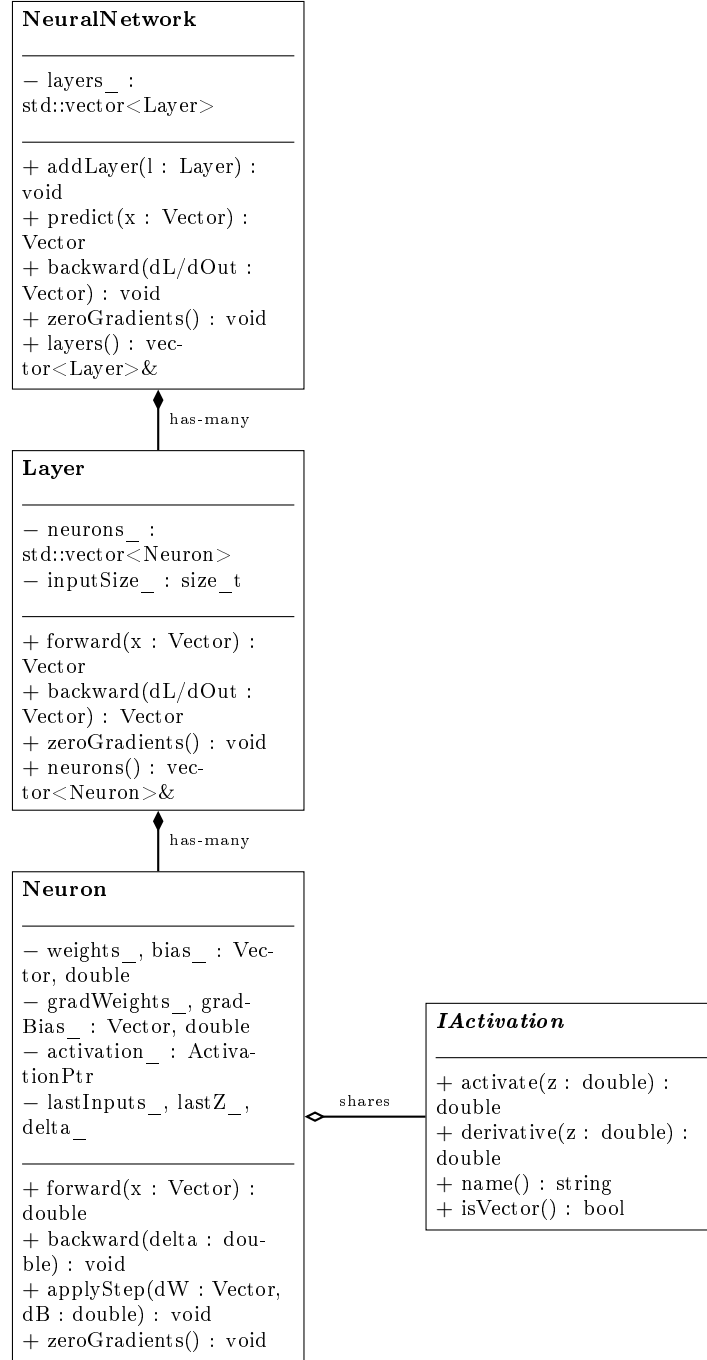


Figure 1: Core Composition Backbone: **NeuralNetwork** $\blacklozenge \rightarrow$ **Layer** $\blacklozenge \rightarrow$ **Neuron** $\diamond \rightarrow$ **IActivation**. Lifetime flows downward; activation objects are shared and immutable.

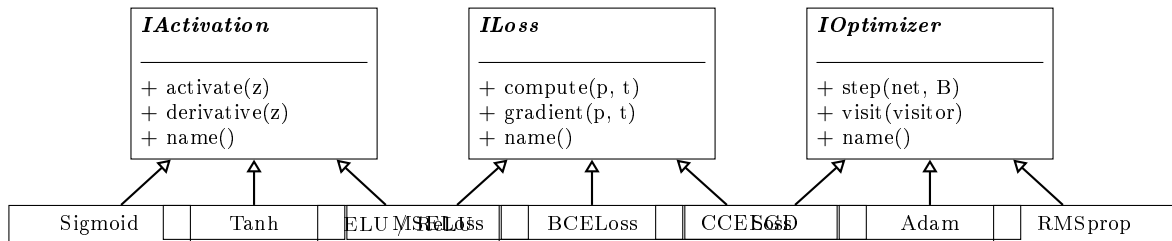


Figure 2: Strategy Hierarchies for Activations, Losses, and Optimisers. Concrete strategies are instantiated via corresponding Factory classes from runtime string tokens.

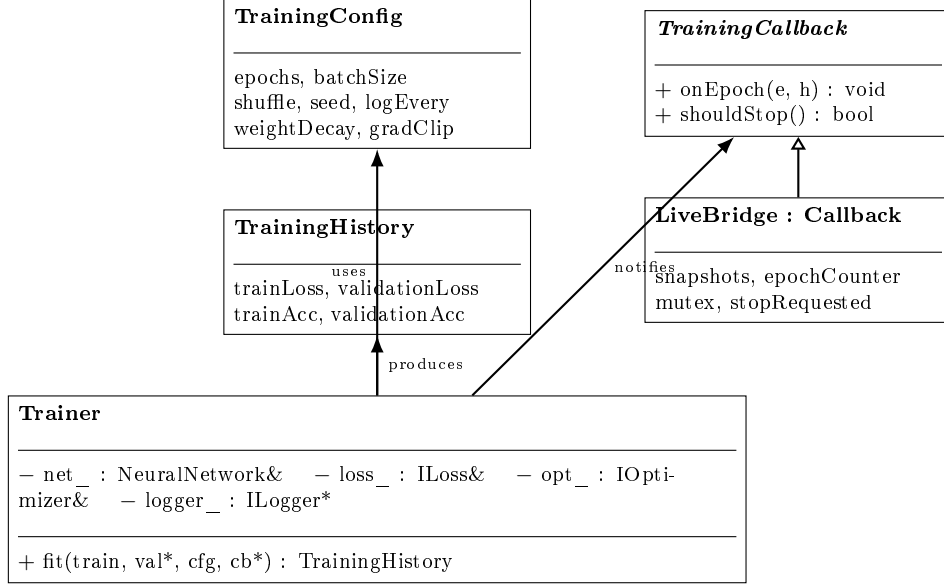


Figure 3: Training Collaboration: **Trainer** aggregates references to the model, loss, and optimiser; executes training using **TrainingConfig**, records **TrainingHistory**, and notifies **TrainingCallback** observers.

9 Dynamic View & Concrete Dry Run Trace

9.1 Per-batch training sequence

The execution order within each batch strictly enforces mathematical invariants:

1. **net.zeroGradients()** zeroes accumulated weight and bias gradients across all layers.
2. For each sample:
 - **net.predict(x)** executes forward propagation, caching inputs $a^{[l-1]}$ and pre-activations $z^{[l]}$.
 - **loss.gradient(pred, target)** computes output error derivative $\nabla_{\hat{y}} \mathcal{L}$.
 - **net.backward(dL/dOut)** propagates deltas backward, accumulating gradients: $\mathbf{g}_W += \delta \mathbf{a}^T$, $\mathbf{g}_b += \delta$.
3. **Regularisation:** If $\lambda > 0$, add L_2 weight penalty to accumulated gradients.
4. **Grad Clipping:** If enabled, evaluate $\|\mathbf{g}\|_2$; scale down if norm exceeds threshold.
5. **optimizer.step(net, B):** Optimiser updates weights using averaged gradients $\frac{1}{B} \mathbf{g}$. Crucially, weights are read before mutation, ensuring subsequent layer backward passes are mathematically untainted.

9.2 Concrete mathematical trace for 1 dry run

We trace a complete forward and backward step on a $2 \rightarrow 2 \rightarrow 1$ network solving XOR.

- **Sample:** $\mathbf{x} = [0.0, 1.0]^T$, Target $y = 1.0$.
- **Hidden Layer 1 (tanh):** $\mathbf{W}^{[1]} = \begin{bmatrix} 0.5 & -0.2 \\ -0.4 & 0.8 \end{bmatrix}$, $\mathbf{b}^{[1]} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}$.
- **Output Layer 2 (σ):** $\mathbf{W}^{[2]} = \begin{bmatrix} 0.7 & -0.5 \end{bmatrix}$, $b^{[2]} = 0.2$.

- **Optimiser:** RMSprop with $\eta = 0.1$, decay $\alpha = 0.9$, $\epsilon = 10^{-8}$, initial second moments $v_0 = 0$.

9.2.1 Step 1: Forward Propagation

$$\begin{aligned}
\mathbf{z}^{[1]} &= \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]} = \begin{bmatrix} 0.5(0) - 0.2(1) + 0.1 \\ -0.4(0) + 0.8(1) - 0.1 \end{bmatrix} = \begin{bmatrix} -0.1000 \\ 0.7000 \end{bmatrix} \\
\mathbf{a}^{[1]} &= \tanh(\mathbf{z}^{[1]}) = \begin{bmatrix} \tanh(-0.1000) \\ \tanh(0.7000) \end{bmatrix} = \begin{bmatrix} -0.0997 \\ 0.6044 \end{bmatrix} \\
z^{[2]} &= \mathbf{W}^{[2]}\mathbf{a}^{[1]} + b^{[2]} = 0.7(-0.0997) - 0.5(0.6044) + 0.2 = -0.1720 \\
a^{[2]} &= \sigma(z^{[2]}) = \frac{1}{1 + e^{0.1720}} = 0.4571 \\
\mathcal{L}_{\text{MSE}} &= \frac{1}{2}(a^{[2]} - y)^2 = \frac{1}{2}(0.4571 - 1.0)^2 = 0.1474
\end{aligned}$$

9.2.2 Step 2: Backward Propagation

$$\begin{aligned}
\sigma'(z^{[2]}) &= a^{[2]}(1 - a^{[2]}) = 0.4571(1 - 0.4571) = 0.2482 \\
\delta^{[2]} &= (a^{[2]} - y) \cdot \sigma'(z^{[2]}) = (-0.5429) \cdot 0.2482 = -0.1347 \\
\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[2]}} &= \delta^{[2]}(\mathbf{a}^{[1]})^T = -0.1347 \cdot [-0.0997, 0.6044] = [0.0134, -0.0814] \\
\frac{\partial \mathcal{L}}{\partial b^{[2]}} &= \delta^{[2]} = -0.1347
\end{aligned}$$

Propagating delta to hidden layer:

$$\begin{aligned}
\tanh'(\mathbf{z}^{[1]}) &= 1 - (\mathbf{a}^{[1]})^2 = \begin{bmatrix} 1 - (-0.0997)^2 \\ 1 - (0.6044)^2 \end{bmatrix} = \begin{bmatrix} 0.9901 \\ 0.6347 \end{bmatrix} \\
\boldsymbol{\delta}^{[1]} &= ((\mathbf{W}^{[2]})^T \delta^{[2]}) \odot \tanh'(\mathbf{z}^{[1]}) = \begin{bmatrix} 0.7(-0.1347) \cdot 0.9901 \\ -0.5(-0.1347) \cdot 0.6347 \end{bmatrix} = \begin{bmatrix} -0.0934 \\ 0.0428 \end{bmatrix} \\
\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[1]}} &= \boldsymbol{\delta}^{[1]}\mathbf{x}^T = \begin{bmatrix} -0.0934 \\ 0.0428 \end{bmatrix} [0.0, 1.0] = \begin{bmatrix} 0.0 & -0.0934 \\ 0.0 & 0.0428 \end{bmatrix}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[1]}} = \begin{bmatrix} -0.0934 \\ 0.0428 \end{bmatrix}
\end{aligned}$$

9.2.3 Step 3: RMSprop Parameter Update

For output weight $W_{1,2}^{[2]}$ (prior value -0.5 , gradient $g = -0.0814$):

$$\begin{aligned}
v_1 &= \alpha v_0 + (1 - \alpha)g^2 = 0.9(0) + 0.1(-0.0814)^2 = 0.0006626 \\
W_{1,2}^{[2]\text{new}} &= -0.5 - \frac{\eta}{\sqrt{v_1 + \epsilon}}g = -0.5 - \frac{0.1}{\sqrt{0.0006626 + 10^{-8}}}(-0.0814) = -0.5 + 0.3162 = -0.1838
\end{aligned}$$

The adaptive learning rate applies a corrective step of $+0.3162$, shifting the weight rapidly toward minimizing the classification error.

10 Verification and Test Results

MiniANN maintains an automated test harness covering all mathematical and architectural components. All tests execute in < 10 seconds.

Test Executable	Tested Functionality	Observed Verification Status
gradient_check	Analytic vs. centered finite-diff	Relative error $3.4 \times 10^{-11} < 10^{-7}$; PASS
test_basic	Activations, serialization round-trip	Round-trip diff = 0.0; PASS
test_choices	8 activations, 4 optimisers, 3 losses	Analytic gradients match numerical diff; PASS
test_training	Multi-optimizer descent & regularisation	Loss descends for SGD, Momentum, Adam, RMSprop; PASS
test_correctness	XOR, AND, OR convergence	100% classification accuracy; PASS
test_callbacks	Observer hooks, early stopping	Early stop at patience threshold; PASS
test_patterns	All 9 GoF design patterns	Strategy, Visitor, Composite, Chain verified; PASS
test_viz3d_math	3D camera projection & layout	Ray picking, orbit bounds, edge budgeting; PASS

Table 3: MiniANN Verification Suite and Test Results

11 Minimal Usage Example

The following self-contained listing demonstrates training XOR from scratch in approximately 30 lines:

```

1 #include "miniann/miniann.hpp"
2 #include <random>
3
4 using namespace miniann;
5
6 int main() {
7     std::mt19937 rng(42);
8     Dataset data = makeXorGate();
9
10    NeuralNetwork net;
11    net.addLayer(Layer(4, 2, ActivationFactory::create("tanh"), rng, WeightInit
        ::Xavier));
12    net.addLayer(Layer(1, 4, ActivationFactory::create("sigmoid"), rng,
        WeightInit::Xavier));
13
14    auto loss = LossFactory::create("mse");
15    auto opt = OptimizerFactory::create("rmsprop", 0.01);
16    ConsoleLogger logger;
17
18    Trainer trainer(net, *loss, *opt, &logger);
19    TrainingConfig cfg;
20    cfg.epochs = 1500;
21    cfg.batchSize = 1;
22    cfg.weightDecay = 0.0001;
23    cfg.gradClip = 1.0;
24
25    TrainingHistory h = trainer.fit(data, nullptr, cfg);
26    CSVLossExporter::exportHistory(h, "xor_loss.csv");
27    ModelSerializer::save(net, "xor.model");
28
29    Vector pred = net.predict({1.0, 0.0}); // Outputs ~1.0
30    return 0;
31 }
```

12 Conclusion

MiniANN demonstrates that a complete neural-network simulation laboratory — dense layers, eight activations, three loss functions, four optimisers, regularisation, gradient norm clipping, metrics, file persistence, terminal visualization, HTML reports, and an interactive desktop workbench featuring both 2D analytics and hardware-accelerated 3D OpenGL network rendering — fits cleanly into dependency-free C++17/C++20 when structured around disciplined object-oriented programming.

The strict encapsulation of mathematical invariants, coupled with flexible GoF design patterns (Strategy, Observer, Factory, Builder, Chain of Responsibility, Visitor, Composite, Façade, and Bridge), produces an architecture that is simultaneously mathematically verifiable, highly extensible, and straightforward to inspect.